

Milestone 4

TLS & Deployment

The backend enforces TLS at multiple layers: the PostgreSQL connection requires `sslmode=require`, Flask session cookies are flagged `Secure`, `HttpOnly`, `SameSite=Strict` and an HSTS header (Strict-Transport-Security) is added to every response. The frontend and backend are served together through a single Flask process that handles both API routes and static file delivery.

Custom Watch Paths

A `watch_config` table stores per-agent custom paths as JSONB. The backend merges these with a hardcoded set of default paths (`/boot`, `bin`, `sbin`, etc, `/root`) at query time rather than persisting the defaults, thus they can never be accidentally removed. Admins update custom paths for a specific agent via `POST /api/config` (session-authenticated) and agents fetch their effective path list via `GET /api/config` (API-key-authenticated). On the agent side, paths are also cached locally in `/etc/hashiko/watch.conf` and refreshed from the server on each run.

Per-Agent Hash Organisation

Hash records store an `agent_id` field (derived from `user@hostname` (machine-id) on the agent) and the `hashes` column is JSONB keyed by directory path. The frontend groups records by agent: an agent grid view lists all known agents and clicking one opens a detailed view showing that agent's hash history with per-path change highlighting computed by `detect_hash_changes`.

REGISTER_TOKEN and HASHIKO_API_URL

Agents no longer ship with a hardcoded API key. On first run, the agent reads `HASHIKO_API_URL` (from `/etc/hashiko/api_url` or the environment variable) and `REGISTER_TOKEN` (from the environment), then calls `POST /api/register` to exchange the one-time token for the permanent `AGENT_API_KEY`, which is persisted to `/etc/hashiko/api_key` (mode 0600) for subsequent runs. The backend only enables the `/api/register` endpoint when `REGISTER_TOKEN` is set in its environment.